

Residência Tecnológica - Entrega Parcial

Squad 1 - Gráfica Máxima

Álef Davi Alves de Carvalho, Henrique Dorneles de Souza, Ilan Vítor Ramos Araújo de Macedo, João Victor Silva Araujo, João Victor da Silva Costa, Kauã Duarte Oliveira, Matheus Lunguinho Matias, Ana Vitoria Alves Pinheiro e Paulo Daniel de Moraes Ferreira.

Curso de Análise e Desenvolvimento de Sistemas e Ciências da Computação -
Universidade Católica de Brasília (UCB) - Campus Ceilândia

RESUMO: O seguinte documento visa documentar todo o escopo de trabalho sobre a problemática enfrentada pela empresa Gráfica Máxima, documentando o processo de desenvolvimento do sistema para gerenciamento de orçamentos no qual visa substituir as planilhas da empresa, utilizando métricas de engenharia, qualidade e teste de software.

Descrição:

Empresa: Gráfica Máxima

Instrutor(a): +55 61 9 8281-3783 (michel22assis@gmail.com)

Contato: +55 61 9 9134-7575

Desafio (completo): Este documento detalha o espaço de trabalho para a **Gráfica Máxima**, cujo principal desafio é a transição de um sistema de orçamentos baseado em planilhas eletrônicas, sujeito a falhas e inconsistência, para uma plataforma de gerenciamento mais consistente. O squad desenvolverá uma solução robusta utilizando rigorosas métricas de **Engenharia de Software, Qualidade e Testes**, garantindo a mitigação de erros operacionais e a otimização do fluxo financeiro da empresa.

Tecnologias:

- Frontend: React.js
- Backend: JavaScript e NODE.js
- Banco de Dados: MySQL (MariaDB)
- Infraestrutura: GitHub, Scrum Lite

1 - Definição do problema e Personas

1.1 - Problema (250 Caracteres)

A dependência de planilhas manuais gera lentidão, erros e perda de dados. Esse modelo obsoleto causa desgaste emocional na equipe e retrabalhos, comprometendo a agilidade, a segurança financeira e a confiabilidade das entregas da empresa.

1.2 - Solução (250 Caracteres)

A solução propõe um software para centralizar e organizar os orçamentos em um único ambiente digital. O sistema substitui métodos manuais e desorganizados, eliminando falhas no controle e otimizando a gestão estratégica das informações financeiras

1.3 - Identificar explicitamente onde IA gera valor (automação, predição ou geração de conteúdo)

A integração de Inteligência Artificial no Projeto Gráfica Máxima atua como um catalisador de produtividade e qualidade técnica. A IA é aplicada de forma estratégica em três pilares fundamentais: na geração de conteúdo para acelerar a escrita de documentação técnica e estruturas de código (boilerplate); na automação de processos e garantia de qualidade (testes unitários e CI/CD); e na predição de dados para apoiar o módulo de orçamentação, permitindo análises preditivas sobre

custos de materiais e demanda operacional. Essa abordagem assegura um ciclo de desenvolvimento mais robusto, escalável e alinhado às melhores práticas de engenharia de software.

1.4 - **Personas**

- **A administradora da Gráfica (Dona/Gerente)**

Focado na saúde financeira, segurança dos dados e visão macro do negócio.

Nome: Administradora

Cargo: Gerente Geral / Proprietário

Idade: 28 anos

Dores:

Insegurança de Dados: Medo constante de perder o histórico de orçamentos por falhas em arquivos de Excel.

Falta de Visão de Lucro: Dificuldade em saber rapidamente qual foi o faturamento real do mês e quantos orçamentos foram perdidos.

- **O Coordenador**

Focado no fluxo de trabalho, aproveitamento de material e logística.

Nome: Marcos Souza

Cargo: Coordenador de Produção

Idade: 39 anos

Dores:

Desperdício de Papel: Dificuldade em visualizar rapidamente como aproveitar melhor a folha inteira (66x96cm) para o pedido do cliente.

Falha de Comunicação: Não saber se um orçamento já foi aprovado pelo financeiro para começar a rodar a máquina.

Esquecimento: Perder tempo configurando a máquina sem que o custo de "entrada" tenha sido cobrado corretamente no orçamento.

- O Cliente

Focado em prazos rígidos, profissionalismo e clareza nos custos.

Nome: Letícia Cavalcante

Cargo: Cliente

Idade: 31 anos

Dores:

Incerteza de Prazos: Ela precisa das apostilas para o início das aulas e fica ansiosa sem saber se o orçamento já foi aprovado ou se entrou em produção.

Falta de Documentação: Recebe valores soltos por mensagem de texto, o que dificulta a prestação de contas com o financeiro da empresa dela.

Demora no Retorno: Perde campanhas de marketing porque a gráfica demora 2 dias para calcular o custo de um papel específico ou formato especial.

1.5 - Cenários de Uso e Casos de Borda (Edge Case) de IA:

- **Orçamento rápido:** O usuário seleciona um tipo de papel, define a tiragem (ex: 1000 cartões de visita), o sistema calcula o valor baseado no custo do insumo + margem de lucro.
- **Status de Orçamento:** O funcionário altera o status do pedido de "Pré-Orçamento" ou "Análise" para "Aprovado", e isso dispara o sistema para gerar o PDF com informações de produção.
- **Criação do Orçamento:** O funcionário define as informações dos orçamentos com informações diretas que produzem um documento completo.

1.6 - Entrevistas e Validação de Hipóteses:

- Relatórios de entrevistas com humanos e agentes inteligentes:
 - **Descoberta de pontos de dor:** As planilhas que fazem o manejo dos orçamentos possuem altas margens de erro que podem prejudicar a

produção da gráfica além de não estarem completas e sem cálculos essenciais.

- **Validação do problema:** Ao analisar ferramentas concorrentes foi analisada toda a base necessária do sistema, priorizando a facilidade de uso.

- **Pelo menos uma entrevista com humanos:**

- **Inconsistência:** O orçamento muda dependendo de qual funcionário faz a conta.
- **Perda de Material:** Eles não sabem exatamente quanto sobra de papel e muitas vezes descartam retalhos que poderiam ser usados.
- **Medo da Tecnologia:** "Tentei usar outro sistema antes, mas era muito complexo e eu perdia mais tempo configurando do que produzindo."

2 - Backlog e Engenharia de Requisitos

Épico 1: Motor de Orçamentação (Core)

Foco: A funcionalidade principal do sistema.

- **User Story: Preciso fazer um orçamento na gráfica.**
 - *Prioridade:* Alta
 - **Tarefas:**
 - Modelagem da tabela budgets no MariaDB.
 - Criação da rota POST /budgets no Node.js.
 - Desenvolvimento do formulário de entrada no React (coleta de dados cliente/orçamento).
- **User Story: Preciso fazer o cálculo para saber quanto irá ficar esse orçamento.**
 - *Prioridade:* Alta
 - **Tarefas:**

- Implementação da lógica de cálculo (cálculo de volume x custo de insumos).
 - Criação de endpoint para persistir o valor calculado.
- **User Story: Preciso emitir o orçamento para o cliente.**
 - *Prioridade:* Média
 - **Tarefas:**
 - Implementação de biblioteca de exportação (PDF).
 - Integração para compartilhamento via link/WhatsApp (API).

Épico 2: Gestão de Acessos e Clientes (CRM/Auth)

Foco: Segurança e relacionamento.

- **User Story: Apenas o gerente pode apagar ou remover orçamentos.**
 - *Prioridade:* Alta
 - **Tarefas:**
 - Implementação de JWT (JSON Web Token) para autenticação.
 - Configuração de RBAC (Role-Based Access Control) para distinguir Admin/Sub-Admin.
- **User Story: Quero ter controle dos clientes que já tive para futuras prospecções.**
 - *Prioridade:* Média
 - **Tarefas:**
 - Criação de tabela customers (normalização de dados).
 - Implementação de busca (autocomplete) para evitar reescritura de dados.

Épico 3: Interface e Monitoramento (UX/Dashboard)

Foco: Usabilidade e visão gerencial.

- **User Story: Preciso fazer um orçamento de maneira rápida em um design fluido.**
 - *Prioridade:* Alta
 - **Tarefas:**
 - Estruturação do Layout (Dashboard base).

- Otimização de componentes React para tempo de resposta rápido.
- **User Story: Preciso saber se o orçamento de sábado foi aprovado.**
 - *Prioridade:* Média
 - **Tarefas:**
 - Criação de estados (Status Machine: Aberto, Construção, Finalizado).
 - Filtros dinâmicos na interface (Status board).
- **User Story: Preciso ver o total de orçamentos aprovados no mês passado.**
 - *Prioridade:* Baixa
 - **Tarefas:**
 - Criação de queries SQL agregadoras (SUM, COUNT com GROUP BY).
 - Visualização gráfica (charts) no dashboard.

Épico 4: Infraestrutura e Segurança (Dados)

Foco: Sustentabilidade e integridade.

- **User Story: Definição dos valores dos insumos.**
 - *Prioridade:* Alta
 - **Tarefas:**
 - Criação da tabela supplies (papel, tinta, hora-máquina).
 - CRUD de insumos para o administrador.
- **User Story: Preciso que o orçamento seja armazenado com segurança e backup.**
 - *Prioridade:* Média
 - **Tarefas:**
 - Configuração de rotina de dump do MariaDB.
 - Garantia de transações ACID no banco.
- **User Story: Preciso verificar quem alterou esse orçamento.**
 - *Prioridade:* Média
 - **Tarefas:**
 - Criação de tabela `audit_logs` (registro de alteração).

- Trigger ou Middleware para capturar `user_id` + `timestamp` em cada update.

2.1- Histórias de Usuário e Critérios de Aceite de IA:

Embora a IA atue no ambiente de desenvolvimento, estabelecemos critérios de aceite para a qualidade das entregas geradas:

Geração de Código (Back/Front) O código deve seguir os padrões de *Clean Code* e não conter *Hardcoding* de variáveis. **Code Review:** Revisão manual do desenvolvedor + Análise de *Linting* automática (ESLint).

Criação de Testes (Jest) Os testes devem cobrir 100% das regras de negócio (Setup, Aproveitamento). **Execução:** O build do CI/CD falha automaticamente se a cobertura de testes cair.

Sugestão de Queries SALAs queries devem utilizar índices existentes e evitar `SELECT *`. **Validação:** Uso do comando `EXPLAIN` no banco para validar o plano de execução da query.

- O Projeto Gráfica Máxima adota uma metodologia de desenvolvimento assistido por IA (AI-Driven Development). Neste contexto, a Inteligência Artificial não compõe as funcionalidades de negócio do produto final, mas atua como uma camada crítica de aceleração e garantia de qualidade no ciclo de vida de desenvolvimento (SDLC). A IA é utilizada para reduzir a carga cognitiva, automatizar tarefas repetitivas de codificação e elevar o padrão de segurança e performance do backend e frontend.

3. Banco de Dados e API

O banco de dados foi modelado para garantir a integridade referencial e o suporte às regras de negócio da gráfica.

Entidades e Atributos

- Usuário: **id_usuario** (PK), nome, email, senha_hash, tipo, data_criacao, ativo
- Cliente: **id_cliente** (PK), nome, cpf_cnpj, telefone, email, endereco, data_cadastro
- Orçamento: **id_orcamento** (PK), **id_usuario** (FK), **id_cliente** (FK), data_criacao, data_validade, valor_total, valor_unitario, status, prazo_entrega, observacoes
- Item: **id_item** (PK), **id_orcamento** (FK), **id_papel** (FK), **id_impressao** (FK), tipo_produto, quantidade, largura, altura, valor_unitario, valor_total
- Serviço/Papel/Impressão/Corte/Acabamento: Tabelas de catálogo para consulta de custos e tipos.
- Tabelas Associativas: **Servico_Item**, **Acabamento_Item**, **Item_Corte** (Relacionamentos N:N).
- Histórico_Log: **id_log** (PK), **id_usuario** (FK), tabela_afetada, id_registro, acao, campo_alterado, valor_antigo, valor_novo, data_hora.

Estruturas Relacionadas ao Uso de IA

Embora a lógica operacional do sistema seja determinística, as seguintes entidades foram otimizadas para futuras integrações com modelos de IA (ex: análise preditiva de custos ou recomendação de produtos):

- Histórico_Log: Armazena o comportamento transacional, servindo como base de dados (dataset) para treinar modelos de detecção de anomalias ou otimização de fluxo.
- Orçamento e Cliente: Estruturas de dados enriquecidas que permitirão futuramente realizar *Demand Forecasting* (previsão de demanda) e *Customer Lifetime Value*.

3.1- Design da API e Endpoints

A API seguirá o padrão RESTful para garantir comunicação eficiente entre o frontend (React) e o backend ([Node.js](#)).

- *POST/api/v1/auth/login* Autenticação do usuário/gestor.
- *POST/api/v1/budgets* Cria um novo orçamento (aciona lógica de cálculo).
- *GET/api/v1/budgets/:id* Recupera detalhes de um orçamento específico.
- *GET/api/v1/budgets* Lista orçamentos (filtros por status/cliente).
- *PUT/api/v1/budgets/:id* Atualiza status ou dados de um orçamento.
- *GET/api/v1/customers* Busca ou lista clientes cadastrados.
- *GET/api/v1/supplies* Consulta lista de preços/insumos atualizada.

3.2- Fluxo de Dados e Lógica do Sistema

O fluxo de processamento integra as regras de negócio cruciais para a Gráfica Máxima:

1. **Entrada:** O usuário preenche os dados do cliente e especificações do produto no frontend.
2. **Processamento (Lógica de Backend):**
 - Regra 01 (Aproveitamento): O sistema calcula o rendimento do formato final na folha (ex: 66x96cm) para determinar o custo exato do papel.
 - Regra 02 (Setup): Aplica-se o custo fixo de "entrada de máquina" antes de multiplicar pela quantidade.
 - Regra 03 (Validade): O backend define automaticamente `data_validade = data_criacao + 15 dias`.
 - Regra 04 (Margem): Aplica-se o markup diferenciado sobre setup (fixo) e produção (variável).
3. **Persistência:** Os dados calculados são salvos no MariaDB com transações ACID.
4. **IA-Assistance (Ciclo de Dev):** Durante este fluxo, utilizamos a IA para a escrita dos testes unitários dos cálculos acima e para otimização das queries SQL de busca, garantindo que o tempo de resposta seja baixo.
5. **Retorno:** JSON estruturado enviado ao Front-end para renderização.

3.3-Tratamento de Exceções e Casos de Falha

A robustez do sistema é garantida através de camadas de tratamento de erro:

- Erros de ValidaçãoRetorno HTTP 400 (Bad Request) com mensagem clara (ex: "CPF inválido" ou "Quantidade negativa").
- Falha de Conexão DBRetorno HTTP 503 (Service Unavailable) com log de erro para monitoramento.
- Valores AusentesValidação no backend via *schema validation* antes de processar qualquer fórmula.
- Timeout em Integração e Implementação de *retry pattern* (3 tentativas) antes de reportar erro ao usuário.
- Falhas na IA (Dev)Caso a sugestão de código da IA cause erro em build, o pipeline de CI/CD trava e exige revisão manual do desenvolvedor antes do deploy.

4. Escolha da Stack de Desenvolvimento e IA

O **Projeto Gráfica Máxima** foi desenvolvido com uma arquitetura de camadas (*Layered Architecture*), focando em desacoplamento e facilidade de manutenção. A stack foi escolhida pela alta performance no processamento de dados e pela vasta comunidade de suporte.

Frontend: React + Tailwind CSS Interface reativa, modular e rápida para o usuário final.

Backend: Node.js (Express)Alta capacidade de processamento I/O, ideal para dashboards.

Banco de Dados: MariaDB para estabilidade, robustez e suporte a transações ACID complexas.

Infraestrutura: GitHub para versionamento de código e Scrum Lite para gestão.

Ferramentas de IA (Desenvolvimento Assistido)

A IA não é uma funcionalidade *runtime* do sistema (que depende de APIs externas para funcionar), mas sim um **co-piloto de desenvolvimento (DevOps & Coding)**.

- **Gratuitas:**
 - **Claude:** Autocompletar inteligente e refatoração de código dentro do VS Code.

- **Gemini (Versão Estudantil):** Apoio na geração de documentação técnica, apoio no desenvolvimento e escrita de *testes unitários* (Jest) e depuração de erros complexos (debugging).
- **Pagas (Opcional):**
 - **Nenhuma.**

4.1- Desenho de Arquitetura e Tratamento de Erros

Estrutura de Pastas (Arquitetura em Camadas)

O projeto segue uma estrutura organizada para separar responsabilidades, 2 repositórios, BackEnd e FrontEnd respectivamente:

BackEnd:

/gráfica-maxima-backend

|----- /src

| |--- /config

| |--- /controllers

| |--- /middlewares

| |--- /models

| |--- /routes

| |--- /services

| |--- /utils

| |--- /app.js

|----- /.database

|----- /node_modules

```
|----- /src  
  
|----- /package.json  
  
|----- /package-lock.json  
  
|----- /server.js  
  
|----- /README.md
```

FrontEnd:

```
/gráfica-maxima  
  
|----- /src  
  
|----- /src  
  
|      |--- /App  
  
|      |--- /index.css  
  
|      |--- /main  
  
|----- /node_modules  
  
|----- /public  
  
|----- /package.json  
  
|----- /package-lock.json  
  
|----- /vite.config.js  
  
|----- /README.md  
  
|----- /eslint.config.js
```

Integração com IA (Ciclo de Vida de Software)

A "integração" ocorre no Build Time (durante o desenvolvimento) e não no Runtime (enquanto o usuário usa).

- Pipeline de Qualidade: Scripts automatizados enviam trechos de código para análise estática via IA antes do *commit*.
- Geração de Testes: A IA auxilia na criação de testes de estresse para os cálculos de orçamento, garantindo que a margem de lucro não fique negativa sob nenhuma condição.

Estratégias de Tratamento de Erros e Disponibilidade

Para garantir que o sistema não pare, implementamos as seguintes camadas de defesa:

- Tratamento de Latência:
 - Caching: Uso de cache em memória para consultas frequentes ao catálogo de insumos (evitando acesso constante ao DB).
 - Queries Otimizadas: Índices criados no MariaDB nas colunas de busca (ex: `id_cliente`) para garantir respostas em milissegundos.

Gestão de Timeout e Indisponibilidade:

- Health Checks: Configuração no Docker para reiniciar o container automaticamente caso o Node.js trave (restart policy).
- Timeouts: Definição de tempo limite (timeout) para todas as conexões com o banco de dados; se o DB não responder em 5 segundos, a requisição é encerrada e um erro é logado, evitando que o backend fique "pendurado".

Resiliência (Graceful Failure):

- Todas as rotas críticas são envolvidas em blocos try/catch. Caso ocorra uma falha não mapeada, o sistema retorna um erro genérico amigável ao usuário (500 Internal Server Error) e registra o *stack trace* completo no sistema de logs para que o desenvolvedor possa corrigir posteriormente.

5. Repositório de Prompts (Prompt Ops)

A documentação abaixo detalha a evolução dos prompts utilizados para garantir a precisão do código gerado para o Projeto Gráfica Máxima.

Modelagem de Banco: Regras de negócio da gráfica (ex: aproveitamento de papel, setup de máquina). Estrutura de entidades, atributos e relacionamentos. Problema: A IA criou tabelas redundantes. **Refinamento:** "Normalize o banco de dados em 3ª Forma Normal e crie tabelas associativas para os relacionamentos N:N".

Definição de Backlog: Lista bruta de funcionalidades solicitadas pelo usuário. Backlog priorizado e categorizado em Épicos e Histórias de Usuário. Problema: A priorização estava muito subjetiva. Refinamento: "Classifique as histórias usando a lógica de valor de negócio e dependência técnica".

Arquitetura Técnica: Stack escolhida (React, Node.js, MariaDB). Diagrama de estrutura de pastas e desenho da arquitetura de camadas. Problema: A estrutura sugerida era complexa demais. Refinamento: "Simplifique para uma arquitetura em camadas focada em um MVP, mantendo a escalabilidade"..

Metodologia de Trabalho

O desenvolvimento do Projeto Gráfica Máxima seguiu uma abordagem híbrida, onde a IA atuou como um "Pair Programmer". O fluxo de trabalho foi definido em três etapas cíclicas que estão ocorrendo ou irão ocorrer durante o restante do processo:

1. **Planejamento:** Definição da lógica de negócio e estrutura de dados (sem auxílio de IA).
2. **Codificação Assistida:** Solicitação de estruturas base (boilerplate) e lógica complexa via LLMs.
3. **Verificação Humana (Quality Gate):** Revisão linha a linha do código gerado, execução de testes e integração ao repositório Git.

5.1- Relato do Processo

Metodologia de Trabalho

Até o momento, o Projeto Gráfica Máxima encontra-se na fase de concepção e engenharia de requisitos. O foco não foi a codificação imediata, mas a criação de uma fundação técnica robusta. A IA tem atuado como uma "Consultora de Arquitetura", auxiliando na tradução de ideias abstratas em documentação técnica profissional.

O processo consistiu em:

1. **Levantamento de Requisitos:** Descrição das necessidades da gráfica.
2. **Modelagem de Dados:** Estruturação lógica do MariaDB para garantir que todas as regras de negócio fossem suportadas.
3. **Planejamento de MVP:** Priorização do que é essencial para o sistema funcionar ("Backlog Priorizado"), garantindo foco no valor de negócio.

Impacto das Ferramentas de IA

O uso da IA nesta fase inicial trouxe clareza e agilidade. O impacto foi direto na **qualidade da documentação**:

- **Estruturação:** A IA ajudou a organizar as ideias soltas em um formato de projeto de software aceito pela indústria (Backlogs, DER, Dicionário de Dados).
- **Validação de Ideias:** Ao explicar as regras de negócio para a IA, conseguimos identificar lacunas na lógica de cálculo de orçamentos (como a regra de setup vs. variável) antes de gastar tempo programando.

Dificuldades e Ajustes

A principal dificuldade foi garantir que a IA compreendesse o contexto específico do negócio (o funcionamento de uma gráfica).

- **Ajuste:** Foi necessário realizar um processo iterativo, onde eu (desenvolvedor) atuava como o "especialista no domínio" e a IA como a "especialista em arquitetura", refinando os requisitos em ciclos de feedback até que a estrutura do banco de dados refletisse perfeitamente o problema real da gráfica.

Aprendizado: Aprendemos que a IA não substitui o planejamento humano, mas potencializa a forma como documentamos esse planejamento, tornando o projeto muito mais claro para futuras etapas de desenvolvimento.